

PICOCTF WRITEUP

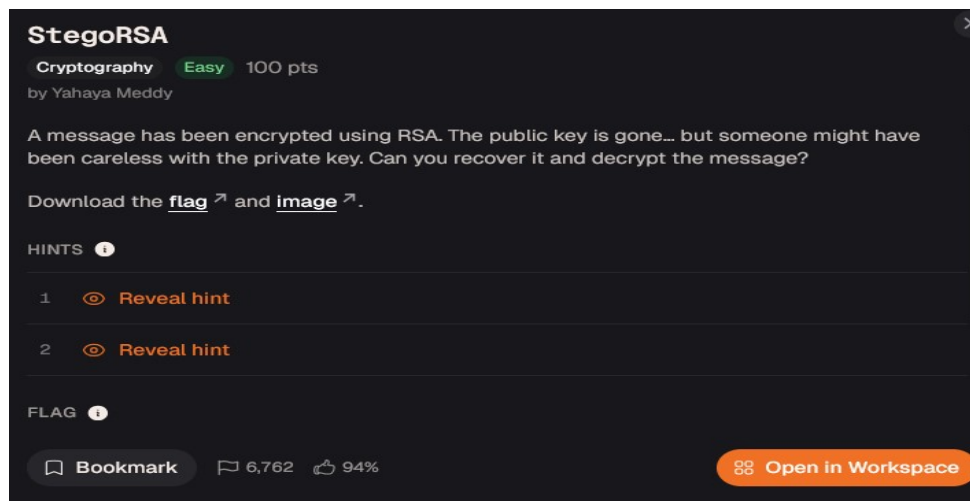
Author :Frank karani

Illustrator

StegoRSA – picoCTF Writeup

Challenge Description

A message has been encrypted using RSA. The public key is gone, but someone may have been careless with the private key. Can you recover the key and decrypt the message?



Files Provided

The challenge provided two files:

- image.jpg
- flag.enc

The goal was to recover the RSA private key and use it to decrypt the encrypted flag.

Step 1: Initial File Analysis

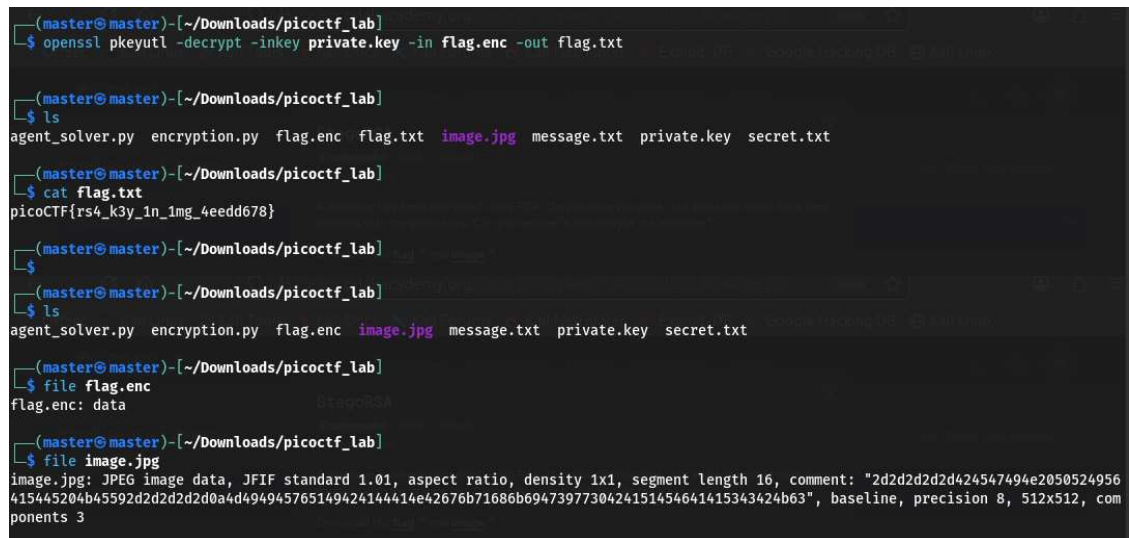
I began by identifying the file types using the `file` command:

```
file image.jpg
file flag.enc
```

The output confirmed that:

- `image.jpg` was an image file.
- `flag.enc` contained encrypted data.

Since the challenge title included the word **StegoRSA**, I suspected that the image might contain hidden information related to RSA.



```
(master@master)-[~/Downloads/picoctf_lab]
$ openssl pkeyutl -decrypt -inkey private.key -in flag.enc -out flag.txt

(master@master)-[~/Downloads/picoctf_lab]
$ ls
agent_solver.py  encryption.py  flag.enc  flag.txt  image.jpg  message.txt  private.key  secret.txt

(master@master)-[~/Downloads/picoctf_lab]
$ cat flag.txt
picoCTF{rs4_k3y_1n_1mg_4eedd678}

(master@master)-[~/Downloads/picoctf_lab]
$ ls
agent_solver.py  encryption.py  flag.enc  image.jpg  message.txt  private.key  secret.txt

(master@master)-[~/Downloads/picoctf_lab]
$ file flag.enc
flag.enc: data

(master@master)-[~/Downloads/picoctf_lab]
$ file image.jpg
image.jpg: JPEG image data, JFIF standard 1.01, aspect ratio, density 1x1, segment length 16, comment: "2d2d2d2d2d424547494e2050524956
415445204b45592d2d2d2d0a4d494945765149424144414e42676b71686b6947397730424151454641415343424b63", baseline, precision 8, 512x512, com
ponents 3
```

Step 2: Extracting Hidden Data

Upon inspecting the image, I discovered a long hexadecimal string embedded within it.

The hexadecimal data appeared to represent an RSA private key encoded in hex format.

Example:

2d2d2d2d2d424547494e2050524956415445204b45592d2d2d2d2d...

Step 3: Converting Hexadecimal to the RSA Private Key

To recover the private key, I converted the hexadecimal string into its original text representation using `xxd`.

```
echo "HEX_DATA_HERE" | xxd -r -p
```

After conversion, the output revealed a valid RSA private key:

```
-----BEGIN PRIVATE KEY-----  
...  
-----END PRIVATE KEY-----
```

Step 4: Saving the Private Key

I saved the recovered key into a file named:

```
nano private.key
```

Then pasted the complete private key and saved the file.

Step 5: Decrypting the Encrypted Flag

With the RSA private key recovered, I used OpenSSL to decrypt the encrypted file.

```
openssl pkeyutl -decrypt \  
-inkey private.key \  
-in flag.enc \  
-out flag.txt
```



```
(master@master)-[~/Downloads/picoctf_lab]  
$ openssl pkeyutl -decrypt -inkey private.key -in flag.enc -out flag.txt  
  
(master@master)-[~/Downloads/picoctf_lab]  
$ ls  
agent_solver.py  encryption.py  flag.enc  flag.txt  image.jpg  message.txt  private.key  secret.txt  
  
(master@master)-[~/Downloads/picoctf_lab]  
$ cat flag.txt  
picoCTF{rs4_k3y_1n_1mg_4eedd678}  
  
(master@master)-[~/Downloads/picoctf_lab]  
$
```

Step 6: Recovering the Flag

After successful decryption, I displayed the contents of the output file:

```
cat flag.txt
```

Output:

```
picoCTF{rs4_k3y_1n_1mg_4eedd678}
```

Flag

```
picoCTF{rs4_k3y_1n_1mg_4eedd678}
```

Lessons Learned

This challenge demonstrated the importance of:

- Proper key management in cryptographic systems.
- Never storing private keys inside publicly accessible files.
- Understanding how steganography can be used to hide sensitive information.
- Combining file analysis, data extraction, and cryptography techniques during investigations.

Even when strong encryption algorithms such as RSA are used, poor operational security can completely compromise confidentiality if private keys are exposed.

Tools Used

- Linux
 - file
 - xxd
 - OpenSSL
 - nano
-

Skills Practiced

- File Analysis

- Cryptography
- RSA Decryption
- Hexadecimal Encoding/Decoding
- Steganography Investigation
- Linux Command Line

THANK YOU ALL

Happy h4ck1ng